EE471C_Lab2_Report

EE471C Lab 2 — Baseband QAM Modem (Report)

Part 1 — In-Lab / Concluding Questions

1. What is the name of the queue used to pass the generated bit sequence from `top_tx.vi` to `top_rx.vi` (i.e., the bit sequence needed to perform error detection)?

`key bit sequence`

2. Describe what happens to the received constellation as you increase noise power in the top level of the transmitter (`top_tx.vi`).

Increasing noise power increases the variance of the AWGN added to each symbol, so each received point `y[n] = sqrt(Es)*s[n] + v[n]` spreads out into a cloud around the ideal constellation point. For particularly noise power, clouds will overlap resulting in symbol/bit errors.

3. Based on your observation of the received signal constellation, explain which modulation scheme (BPSK or QPSK) performs better (on average) in noisy channels, and why. Assume both schemes use the same average symbol energy.

BPSK performs better. At equal average symbol energy `Es` , BPSK's two constellation points (`±1`) are separated by the maximum possible distance (`d_min = 2`), while QPSK's four points (`±1/√2 ± j/√2`) are packed more densely for the same energy ceiling (`d_min = √2`).
Also, QPSK carries 2 bits per symbol whereas BPSK carries 1 -- this results in QPSK's BER being consistently higher as its energy-per-bit is half of that of BPSK.

The BER v SNR plot submitted in Part 1's prelab also corroborates this result -- the BPSK error probability drops off sqrt(2)x faster than QPSK error probability as noise is decreased.

Part 2 — In-Lab Questions

Setup assumed: Packet length = 200 bits, QPSK, Root Raised Cosine, filter parameter α swept 0→1 in steps of 0.20, filter length = 8, capture time = 2 msec.

1. Vary the RRC rolloff factor α from 0 to 1 in increments of 0.20. Observe how the eye diagram on `top_rx.vi` 's front panel changes.

(a) Describe what happens to the opening of the eye as you increase α from 0 to 1.

The eye **opens wider** as α increases. A larger rolloff factor makes the Root Raised Cosine pulse's time-domain tails decay faster and the signal converges more tightly around the ideal constellation points. At α≈0 (near-sinc), the slowly-decaying `1/t` tails leave the eye nearly closed.

(b) Explain what impairment is being added or removed from the received signal as you vary this parameter.

As α **decreases** toward 0, the pulse-shape's time-domain tails decay more slowly, so the composite Nyquist pulse `g(t) = g_rx * g_tx` is more sensitive to any residual sampling-timing error. So a small deviation from the ideal sampling rate/instant will result in more spectral leakage and therefore more inter-symbol interference.

2. Based on [1] and your observations, what is the relation between the excess bandwidth parameter (α) and ISI? Does ISI increase or decrease with increasing α, and why?

ISI decreases as α increases. The rolloff factor directly controls how quickly the pulse shape's time-domain tails decay: higher α → faster decay → less energy from neighboring symbols → less ISI. This is the fundamental time-frequency tradeoff that the rolloff factor parameterizes.

Lab Report — Submission Directions

1. Answer all questions from Parts 1 and 2 of Lab 2 (not pre-lab)

Covered above.

2. Discuss any problems encountered in either Part 1 or Part 2, and how you overcame them

Ran into an issue in Part 1 because my modulate or decode was initially coercing symbol data into an unsigned type. This was causing `-1` symbol to become `255` , resulting in particularly weird data -- noisy data even when noise was set to 0.

Also had some issues with the plugging into `Convolution.vi` in `pulse_shaping.vi`. Upsampling an array (and padding with zeroes) was easy, but LabView requires a the termination of the loop tunnel to be set to "Concatenation" mode -- in order to collapse several `[sample, 0, 0, 0]` arrays into a single array that can be passed into convolve(). Got around this by creating many indicators at each step and issuing breakpoints to triage the type discrepancies I was observing.

3. Generalize `modulate.vi` / `decode.vi` to an arbitrary constellation `C` (given as a 1-D array of complex doubles)

`modulate.vi` generalization:

- Instead of a `case` structure keyed on a `"BPSK"` / `"QPSK"` string, take the constellation `C` itself as an input (array of CDB, length `M`), with `b = log2(M)` bits per symbol.
- **Energy normalization**: compute the constellation's average energy once, `E_avg = (1/M) * sum(|C[m]|^2)`. Divide every element of the constellation by `sqrt(E_avg)` to produce a unit-energy (normalized) version. This factor resolves to sqrt(2) in the specific QPSK case, but generalizes to any PSK constellation.... we can call this normalized array `C_norm`.
- **Symbol mapping**: group the input bit stream into chunks of `b` bits. Then each chunk should be mapped to a binary sum: `index = sum(bit_i) * 2^i`. This replaces the 1-2*bit operation which would only work for 1,-1 on either axis.
- Get symbol `s` by extracting `C_norm[index]`

`decode.vi` generalization:

- For each received symbol `y`, compute the squared Euclidean distance to every scaled candidate point `sqrt(Es)*C_normalized[m]` and identify the index at which the minimum distnace is present. That index corresponds to the exact symbol to decode to.
- Convert that index back to its `b`-bit pattern using the inverse of the computation shown in the third bullet under modulate.vi section above.

4. Are "reduce spectral sidelobes" and "reduce time-domain pulse tails" conflicting or complementary goals for pulse shaping, and why?

Conflicting. This is a direct consequence of the time–frequency duality of the Fourier transform: a pulse cannot be made arbitrarily compact in both the time domain and the frequency domain simultaneously. A signal that is infinite in frequency domain (like sinc) is finite in time domain (mapping to rect).

Wanting **minimal spectral sidelobes** desires a pulse that has a compact frequency response. But that same "sinc" pulse would have a time-domain response that decays as `1/t` with very long tails (infinite). In practice, that needs to be truncated... which ultimately leaves some sensitivity to ISI.

The raised-cosine rolloff factor `α` allows a designer to choose between the conflicting goals

- `α` toward 0 to save bandwidth at the cost of ISI sensitivity

- α toward 1 to gain robustness against ISI at the cost of spectral efficiency.